

编译原理研讨课CACT实验PR002报告

成员组成

第03小组：

杜璵 俞博涵 周波

任务说明

实验目标：

1. 正确运用属性依赖的相关理论，掌握设计AST节点属性的方法；
2. 通过在AST遍历过程中正确引入代码，实现CACT语言的语义分析；
3. 理解语义分析对静态源码检查的作用；
4. 在语义分析的基础上，生成出自己设计的IR实现；
5. 支持IR的打印输出，解释执行。

实验过程

实验内容

1. 根据语义分析和中间代码生成要求修改.g4文件；
2. 从依赖关系出发，设计抽象语法树各个节点属性并将属性插入节点遍历过程代码，完成语义分析实现静态源码检查；
3. 设计自己的IR完成中间代码的生成；
4. 设计解释器完成中间代码的输出；

设计思路

1. 符号表的设计

符号表记录了所有变量和函数的名称、类型、作用域等信息。语义分析依赖这些信息来判断标识符是否合法。程序中的同名变量可能出现在不同的作用域。符号表通过作用域链（`scope_t`）来支持嵌套作用域的变量查找。在表达式求值、函数调用、赋值语句中，语义分析器通过查符号表获取类型信息来判断类型是否匹配。

为此创建了一个支持嵌套作用域的符号表用于管理函数和变量名，对它使用哈希表加速查找，提供支持查找变量定义的方法。最后语法树工作流程为遍历语法树（AST），使用listener模式：遇到变量、函数声明时 → 将其加入符号表；遇到变量、函数使用时 → 查询符号表进行合法性和类型检查；遇到新作用域时 → 创建新的 `scope_t` 实例，进入子作用域；作用域结束时 → 返回父作用域。

2. 语义分析的完成

根据实验讲义上的要求，通过antlr生成的访问语法树的接口，用enter和exit方法定义进出节点时需要进行的语义动作，即对各个节点属性的操作，以此检查测试程序是否符合语义规则。

嵌套工作域的运用非常重要，初始化操作：设立全局作用域（`root_scope`）和当前作用域指针

（`cur_scope`），并预注册内建函数到函数符号表中(如`printf`)。在进入函数、代码块等结构时创建新的作用域节点，并在退出时恢复为父作用域，确保变量和函数在正确的作用域中声明和使用。

设立变量符号表(`var_table_t var_table`)和函数符号表(`func_table_t func_table`)处理对应元素：变量声明与定义处理：在遇到变量或常量定义时，提取基本类型、数组维度和常量属性并检查是否在当前作用域内

重定义，然后构建类型信息并注册到变量符号表中；在遇到函数定义与参数处理时检查函数是否重复定义；提取返回类型和参数列表然后将函数体中的形参被加入到新建的局部作用域中。

完成静态源码检查：(语义检查) 标识符使用检查：对变量使用（如左值 LVal）进行符号表查找，同时传递类型、常量属性等信息用于后续验证。表达式与赋值检查：分析表达式结构并向上构建类型；使用 `OperandCheck()` 检查一元和二元运算是否合法；对赋值语句检查左值是否为常量，左右类型是否一致。函数调用检查：查找函数符号表，验证调用是否存在；检查实参与形参的数量、类型和数组维度是否匹配；报告不一致的调用错误。返回类型一致性验证：检查函数体的最终返回类型是否与函数声明类型一致；检查控制流语句如 `while`、`if`、`return` 的合法性。

错误报告：在上述各种检查中加入错误报告，遇见错误情况时输出1，并且打印错误的语义情况。

3. 中间代码的实现和生成

设计中间代码格式如下：

```
typedef struct{
    IR_op_t IROP;
    cact_basetype_t basetype;
    std::string result;
    std::string arg1;
    std::string arg2;
}IR_code_t;
```

其中 `IROP` 表示操作码，`basetype` 表示返回结果的数据类型，`arg1`、`arg2`、`result` 分别表示两个操作数和返回结果的名字。

为完成中间代码的实现需要对当前已有的文件进行新的补充和修改：

对 `g4` 文件进行新的补充：主要目的是为了更方便中间代码传递相关信息，尤其是在控制流相关语句中的 `label` 的传递；另外也新添加了 `lab` 和 `go` 节点来完成中间代码中有关跳转指令的语义动作。

对中间代码块的类型进行新的定义：因为整个实验中存在有大量的基本定义变量，新创建了文件 `cact_types.h` 头文件存放新定义的类型。同样地，将中间代码中新定义的类型添加到里面，包括有中间代码操作类型 (`IR_op_t`)，语句类型 (`IR_code_t`) 和操作对象类型 (`IR_temp_t`)。

对符号表：为变量和函数形参的符号表项中额外记录了中间代码对象名，从而可以在语义分析中直接根据变量节点使用其对象。

对语义分析程序：在类中管理 `label`、全局/局部/临时变量的生成以保证不重复。在遍历语法分析树时，根据相应的语义动作中加入中间代码块，生成对应的中间代码语句。

4. 解释器的设计和中间代码的输出

基本思路类似于计组 `lab` 中关于 CPU 的设计。通过定义新的参数 `pc` 作为指令执行顺序的量化标准，在打印中间代码结果时将 `pc` 也打印出来，这样就可以观察程序的运行顺序和每一步的对应语义。因为 `pc` 和程序的运行顺序有关，因此在解释器需要进行两次完全的中间代码遍历：先从上到下扫描一遍 `IR` 保存标签对应的地址，将中间代码对应的函数名、标签名映射到它们在 `IRC_array` 中的索引(即 `pc`)，这样就可以得到文本上的中间代码顺序；第二次扫描则是进行输出中间代码的操作，正常情况下按照 `pc` 顺序进行顺序输出即可，每次输出一句后执行 `pc++` 操作。如果出现跳转指令(程序控制流指令)，下一步 `pc` 的值不能顺序得出，这时候只需要通过这一步代码输出结果对应的标签名就可以得到跳转位置的 `pc` 值，即 `(labels[ir.result].begin)`，此时再 +1 就是下一条指令的 `pc` 值。

如何得到需要输出的中间代码：根据前面的叙述可以知道 `pc` 和每一句中间代码对应的标签名有关，而每个标签名由操作码决定，所以通过 `pc` 我们就已经可以确定 `IROP`。获得操作数的值和返回结果的值：因为全局变量和局部变量可以重名，首先需要确定值的作用域，选择通过特殊的标号进行标记：'#' 表示立即数，'@' 表示全局变量，'^' 表示局部变量， '%' 表示临时变量，取值时将前面符号去除即可。得到操作数

的值：通过函数`getValue(std::string name)`得到，如果不是数组元素直接取定义变量`name[0]`的0号位，如果是数组元素，数组元素IR表达式格式设定为`@0<index_expr>`，如果要取每个特定的值下标需要另外计算(假设4字节为单位)，然后再访问对应位置的值。设置结果的值：通过函数`setValue(std::string name, int val, int index)`得到，将结果写入变量表中，写入指定变量名和对应的偏移位置。

代码实现

1. `cact_types.h`文件：

`cact_types.h`主要定义了各种基本类型，以供后续代码文件使用。

`cact_basety_t`定义了变量的基本类型：关于

```
typedef enum {
    BTY_UNKNOWN = 0,
    BTY_VOID,
    BTY_INT,
    BTY_BOOL,
    BTY_FLOAT,
    BTY_CHAR,
    //IR
    BTY_INT_PTR,
    BTY_BOOL_PTR,
    BTY_FLOAT_PTR,
    BTY_CHAR_PTR,
} cact_basety_t
```

`cact_type_t` 结构体用于全面描述一个变量的类型，包括三个成员：一个布尔值 `is_const`，标识该变量是否为常量；一个 `cact_basety_t` 枚举类型 `basety`，用于记录变量的基本类型；一个数组维度列表 `arrdims`，当变量是数组时，该字段保存其维度信息，否则为空。这一结构在变量声明、表达式类型判断等环节都起到核心作用。

```
typedef std::vector<uint32_t> arrdims_t;
typedef struct {
    bool        is_const;
    cact_basety_t basety;
    arrdims_t   arrdims;
} cact_type_t;
```

表达式类型 `cact_expr_t` 定义了一棵表达式语法树中的一个节点。它包含的成员有：`op` 表示该表达式节点的操作符，是 `cact_op_t` 枚举类型；`basety` 表示表达式的求值结果类型；`subexprs` 是子表达式的集合，用于表示一元、二元或多元操作结构；`arrdims` 则表示如果该表达式表示的是一个数组变量或数组元素，它的维度信息是什么。所有表达式通过智能指针 `cact_expr_ptr` 管理，确保在多处共享表达式结构时资源安全释放。

```

struct cact_expr;
typedef std::shared_ptr<cact_expr> cact_expr_ptr;
//指向子表达式的指针数组
typedef std::vector<cact_expr_ptr> subexprs_t;
typedef struct cact_expr{
    cact_op_t      op;
    cact_basety_t  basety;
    subexprs_t     subexprs;
    //只有当op为OP_BASE且basety为int时才会使用
    int            int_result;
    //只有当op为OP_ARRAY时使用
    arrdims_t      arrdims;
}cact_expr_t;

```

中间代码宏IR_GEN引入了一套描述中间表示的结构。IR_op_t 是枚举类型，列举了所有可能的中间指令类型，如 IR_ADD、IR_SUB、IR_ASSIGN 等，这些操作码构成了解释器执行的核心逻辑。IR_code_t 结构体用于表示一条完整的中间代码指令，它包含了操作码 IRop、该操作对应的数据类型 basety、结果变量名 result 以及两个操作数 arg1 和 arg2。这是解释器运行时读取和执行的基本单位。

```

#ifdef IR_GEN
typedef enum{
    IR_UNKNOWN=0,
    IR_LABEL,
    IR_FUNC_BEGIN,
    IR_FUNC_END,
    IR_PARAM,
    IR_CALL,
    IR_RETURN,
    IR_ASSIGN,
    IR_ADD,
    IR_SUB,
    IR_MUL,
    IR_DIV,
    IR_MOD,
    IR_AND,
    IR_OR,
    IR_NEG,
    IR_NOT,
    IR_BEQ,
    IR_BNE,
    IR_BLT,
    IR_BGT,
    IR_BLE,
    IR_BGE,
    IR_J,
    IR_G_ALLOC,    //全局变量声明
    IR_L_ALLOC,    //局部变量声明
}IR_op_t;

typedef struct{

```

```

    IR_op_t IRop;
    cact_basety_t basety;
    std::string result;
    std::string arg1;
    std::string arg2;
}IR_code_t;

typedef struct{
    cact_basety_t basety;
    bool is_const;
    size_t length;    //数组
    size_t offset;
}IR_temp_t;
#endif

```

TypeUtils 是一个工具类，封装了多个类型映射表。str_to_basety 将源代码中的字符串类型（如"int"）映射为 cact_basety_t；basety_to_str 则是反向映射，用于调试输出；basety_to_size 提供每种基本类型在内存中占用的字节数。除此之外，该类还维护了从操作符字符串到操作枚举（str_to_op），以及从操作符枚举到 IR 操作码（op_to_IRop）的转换映射。在启用 IR 生成时，IRop_to_str 则用于将操作码转换为人类可读的字符串，便于调试输出和 IR 打印：

```

#ifdef IR_gen
std::map <cact_op_t, IR_op_t> op_to_IRop{
    {OP_NEG, IR_NEG},
    {OP_NOT, IR_NOT},
    {OP_ADD, IR_ADD},
    {OP_SUB, IR_SUB},
    {OP_MUL, IR_MUL},
    {OP_DIV, IR_DIV},
    {OP_MOD, IR_MOD},
    {OP_LEQ, IR_BLE},
    {OP_GEQ, IR_BGE},
    {OP_LT, IR_BLT},
    {OP_GT, IR_BGT},
    {OP_EQ, IR_BEQ},
    {OP_NEQ, IR_BNE},
};

std::map <IR_op_t, std::string> IRop_to_str{
    {IR_LABEL, "Label"},
    {IR_FUNC_BEGIN, "Func Begin"},
    {IR_FUNC_END, "Func End"},
    {IR_PARAM, "Param"},
    {IR_CALL, "Call"},
    {IR_RETURN, "Return"},
    {IR_ASSIGN, "ASSIGN"},
    {IR_ADD, "ADD"},
    {IR_SUB, "SUB"},
    {IR_MUL, "MUL"},
    {IR_DIV, "DIV"},
    {IR_SLL, "SLL"},

```

```

    {IR_SRA,      "SRA"},
    {IR_MOD,      "MOD"},
    {IR_AND,      "AND"},
    {IR_OR,       "OR"},
    {IR_NEG,      "NEG"},
    {IR_NOT,      "NOT"},
    {IR_BEQ,      "BEQ"},
    {IR_BNE,      "BNE"},
    {IR_BLT,      "BLT"},
    {IR_BGT,      "BGT"},
    {IR_BLE,      "BLE"},
    {IR_BGE,      "BGE"},
    {IR_J,        "J"},
    {IR_G_ALLOC,  "G_Alloc"}, //全局变量声明
    {IR_L_ALLOC,  "L_Alloc"}, //局部变量声明
};
#endif

```

2. 符号表设计 (SymbolTable.h文件)

class SymbolTable为本次实验定义的符号表类，该类的主要作用是记录各标识符的信息并处理作用域的范围。其提供的主要方法是对符号表的查找。由于变量有嵌套作用域，函数有形参，因此我们将变量和函数的符号表分开实现如下：变量表设计

map数据结构具有key不重复性的特征，且变量在声明时不允许重名，所以适合用作表的检查。此外，由于不关注变量的声明顺序，使用基于hash的unordered_map可以提高查找的性能，符号表表项数据结构的定义如下，

```

typedef std::unordered_map <name_scope_t, var_symbol_item_t, hash_utils> \
    var_table_t;

```

为了使 name_scope_t 能被用于哈希容器（如 unordered_map）中，代码定义了一个结构体 hash_utils，重载了函数调用运算符 operator()，用于为 name_scope_t 计算哈希值。该函数将字符串哈希和作用域指针的哈希做异或运算，从而生成唯一键。

```

C++
typedef struct name_scope{
    std::string name;
    scope_t *scope_ptr;

    bool operator==(const name_scope &other) const{
        return (other.scope_ptr == scope_ptr) && (other.name == name);
    }
} name_scope_t;

```

指针scope_ptr指向该变量从属的作用域。按照CACT规范，程序中的作用域呈树状，也就是以全局作用域为根节点，函数和全局block块作为一级子节点，而它们的子节点又组成了二级子节点.....所以我们可以使用多叉树来组织嵌套作用域的结构，树的每一个节点便表示一个作用域。与此同时，为了实现变量在嵌套作用域中的查找，每个子节点还包含一个指向其父节点的指针，用于向上级作用域查找变量。

`var_symbol_item_t`表示变量表中的表项，同时也是`map`中存储的`value`值。其中`cact_type_t`的定义位于`cact_types.h`头文件中，该类型包含了常量和数组的信息。在启用了中间代码生成（`IR_GEN`宏定义）时，还包含一个`IR_name`字段，用于记录该变量在`IR`中的名称，用以后续中间代码生成和解释执行。

```
typedef struct var_symbol_item{
    cact_type_t type;
#ifdef IR_GEN
    std::string IR_name;
#endif
} var_symbol_item_t;
```

同时，由于作为`unordered_map`中键值的`name_scope`不是基本类型，而是自定义的结构体，因此需要重载`unordered_map`中使用的`hash`函数，即定义下述`hash_utils`结构体，使用`operator()`运算符来重载。

```
C++
struct hash_utils{
    size_t operator()(const name_scope_t &tmp) const {
        return (std::hash<std::string>()(tmp.name) ^ std::hash<scope_t *>()
(tmp.scope_ptr));
    }
};
```

函数表设计

函数表的设计与变量表的设计类似。由于函数内不允许嵌套函数，函数只能在全局作用域里声明和定义，因此`unordered_map`中只需用函数名作为`key`。而函数名的类型属于`string`基本类型，因此可以直接使用其对应的`hash`方法，不必像变量表那样额外定义`hash_utils`。函数表表项数据结构的定义如下：

```
typedef std::unordered_map <std::string, func_symbol_item_t> \
    func_table_t;
```

`func_symbol_item_t`表示函数表表项，是`unordered_map`中存储的`value`值。函数的返回值类型只能为基本类型，具体定义为：

```
C++
typedef struct func_symbol_item{
    cact_basety_t ret_type;
    fparam_list_t fparam_list;
}func_symbol_item_t;
```

上述的`fparam_list`表示函数形参列表，列表的每一项由形参名和存储形参信息的`fparam_item_t`类型条目构成，每个条目包含了形参的基本类型`type`和引用的顺序`order`。由于`fparam_item_t`是一个`map`，而`map`在排序时默认按照字典序排，而我们要求形参应该按照在引用中出现的先后顺序进行排列，所以使用`operator<`重载运算符来指定排序方式。

```

C++
typedef std::map<std::string, fparam_item_t> fparam_list_t;
typedef struct fparam_item{
    cact_type_t type;
    int order;

    bool operator<(fparam_item const &item) const{
        return order < item.order;
    }
} fparam_item_t;

```

符号表查找

SymbolTable 是符号表的管理者，包含了三个主要成员变量。首先是 `root_scope`，它指向全局作用域，是所有作用域链的起点。其次是 `var_table` 和 `func_table`，分别是变量表和函数表。这个类还提供了一个符号表查找方法 `deepfind()`，用于从当前作用域向上逐层查找指定名称的变量。其实现方式是，给定变量名和当前作用域指针，尝试构造 `name_scope_t` 并在 `var_table` 中查找；若未找到，则向父作用域递归查找，直到根作用域为止或查找到目标变量。`deepfind()` 是核心函数，用于处理变量引用时的作用域解析问题。

```

//find只能查找特定作用域的var，用于声明检查
//deepfind将向上搜索
var_table_t::iterator deepfind(std::string name, scope_t *scope_ptr){
    auto end_flag = var_table.end();
    while (scope_ptr != NULL){
        auto iter = var_table.find((name_scope){.name=name,
        .scope_ptr=scope_ptr});
        if (iter != end_flag){
            //找到该变量
            return iter;
        }
        else
            scope_ptr = scope_ptr->parent;
    }
    //如果最终未找到，返回end_flag用于判断
    return end_flag;
}

```

3. 语义分析

头文件设计 (SemanticAnalysis.h)

该头文件主要定义了SemanticAnalysis类，它从CACTListener父类继承。在该类中，我们分别声明`root_scope`和`cur_scope`，用来表示根作用域（全局作用域）和当前所处的作用域。同时我们也在该类中声明各种语义动作：

- 添加内联函数。由于测试用例中直接调用`print_double`等内联函数且未声明，因此我们需要将这些内联函数自行填入符号表中，否则测试样例在调用这些函数时会出现报错。

- 添加进入和退出.g4文件中定义每个非终结符节点的行为，主要是对各节点综合属性和继承属性进行语义操作。在代码实现上，需要获得对应RuleContext指针，再调用enter和exit方法进行定义。
- 进行语义检查，检查的内容主要可以分为函数、表达式&初值和作用域三个方面。需要检查函数中的函数注册、参数校验、递归和返回值，也需要检查表达式中的多维数组的赋值与定义，变量类型和子表达式间的操作，同时还要创建和维护嵌套作用域。
- 语义分析时的动作函数可以借助模板函数实现非常量的变量声明和常量声明，即template来同时实现这两种情况。如在SemanticAnalysis.h中声明的：

```
template <typename T1,typename T2>
void enterConst_Var_Decl(T1 *ctx,std::vector<T2*> def_list);

template <typename T1>
void enterConst_Var_Def(T1 *ctx);

//is_const区分constDef和varDef添加到变量表的不同类型
template <typename T1>
void exitConst_Var_Def(T1 *ctx, bool is_const);
```

CACT.g4在语义分析中做出的修改

使用antlr提供的locals指令可为各结点添加属性，由于在添加属性时会用到之前自定义的结构类型，因此要在头文件中加入# include "cact_types.h"。在各节点定义并相互传递属性时语义分析的要求。

- 常量和变量的声明 在常量声明中，根据语法规则constDecl : CONST bType constDef (COMMA constDef)* SEMICOLON ;可知，被声明的常量类型定义在bType中，因此与类型相关的属性需要传递到constDef节点中。此外，constDef节点可能有多个，这些节点都需要来自bType节点的类型属性。所以，bType和constDef节点中的属性定义如下所示：

```
bType
  locals[
    cact_basety_t basety,
    std::vector<cact_basety_t*> passTo,
  ]
  : 'int'
  | 'float'
  | 'char'
  ;
constDef
  locals [
    cact_basety_t basety,
    std::vector<uint32_t> arraydims,
    std::string name,
    cact_type_t type,
  ]
  : IDENT arrayDims '=' constInitVal
  ;
```

bType节点和constDef节点都添加了表示被声明变量类型的属性basety，其中constDef中的basety是由bType中的basety继承而来。bType中还定义了passTo属性，它是一个指向后面各个constDef节点中basety的指针数组，使用vector容器来实现。容器中的指针数量即为后面constDef的数量，可以在进入父节点（constDecl）时得到。遍历语法树，当离开bType节点，即将进入constDef时，语义动作定义成用bType节点获得的basety填充其vector容器中所指向的后续constDef节点basety属性。通过这种方式，我们可以在进入constDef节点时提前获得同一层级的btype节点提供的类型信息，避免在退出上层节点时再判断constDef变量声明的合法性，从而减少子节点的重复遍历，提升性能。

此外，在constDef节点中还定义了维度（数组），常量名和常量的type，在离开constDef节点时要根据这些属性把新声明的常量加入到符号表中适当的位置中。

- 多维数组初始化

然后继续考虑语法规则constDef : Ident arrayDims ASSIGN constInitVal;，constInitVal节点中的basety属性从constDef处继承，继承方式类似于bType和constDef之间的继承方式，区别是constDef表达式中只存在一个constInitVal。同时，向constInitVal节点中添加的属性，其语义动作要能够检查多维数组的初始化。

```
constInitVal
  locals[
    //自顶向下继承
    cact_basety_t basety,
    //维度数组指针，值依次是从最外层到最内层的维度
    std::vector<uint32_t> *dims_ptr,
    //维度索引，最内层为0
    uint16_t dim_index,
    //表征是否最顶层，只有index为1，且为最顶层才允许以平铺列表初始化
    bool top,

    //IR
    //value_list为所有子元素拼接，空位置用#占位
    std::string value_list,
  ]
  : constExp
  | '{'(constInitVal(',' constInitVal)*)?}'
  ;
```

考虑到constInitVal推导出的是多维数组初始化的情形，定义属性dims_ptr，它是一个数组指针，指向存储被声明数组各个维度的数组（用vector实现），该vector所记录的各个维度的值在进入constDef节点时由arraydims传递到constInitVal中。为了区分多维数组初始化中的嵌套定义和平铺列表式定义，引入dim_index和top两个自顶向下的集成属性，前者表示嵌套的{}的层数，可以根据层数判断多维数组初始化的定义方式，后者表示此时constInitVal节点推出的{}是否为最顶层的，仅当该节点由constDef推出时top为True。如果top为TRUE且dim_index为1，则认为可能是平铺列表初始化，与维度数组各维长度的乘积进行比较，否则只能与维度数组的特定维度比较。通过上述几种属性的传递，可以逐层退出constInitVal节点时自底向上的判断多维数组各维度的合法性。特别的，由于自底向上的维度检查不易确定终点，在离开constDef节点时还需要检查constInitVal最顶层数组的dim_index并与声明的多维数组进行比较，从而保证数组初始化的正确性。

- 函数的声明

考虑语法规则 `funcDef : funcType Ident LeftParen funcFParam? (COMMA funcFParam)* RightParen block;`, `funcType`中定义了函数返回值类型, `funcFParam`中定义了函数形参, 其数量 ≥ 0 。由于形参列表需要从各个 `funcFParam`中获得, 并且还要传递到 `block`中, 以便检查函数作用域中是否出现和形参重名的变量声明, 因此在进入 `funcDef`节点时需要把该节点中的 `fparam_list`属性传递到它的各个 `funcFParam`子节点中, 由它们进行插入, 而 `funcDef`只负责管理它的 `fparam_list`属性。由此又可以得出每个 `funcFParam`中的属性需要包含指向 `fparam_list`的指针, 即 `fparam_list_ptr`, 同时也要记录该节点所生成形参的在列表中的顺序, 用 `order`进行标记。

对于 `block`, 由于 `block`节点需要从其父节点 `funcDef`中继承形参列表, 因此也需要向其添加 `fparam_list_ptr`的属性。为了检查函数返回值的类型与函数定义时的返回值类型是否相同, `block`节点还需定义一个 `ret_type`属性, 用来和 `funcType`中的 `basety`比较。在退出 `funcDef`节点时, 已经知晓 `funcType`和 `block`中的 `basety`和 `ret_type`, 就可以进行函数返回值类型是否和定义相同的语义检查。

语义分析主文件(SemanticAnalysis.cpp)

- 表达式&初值
 - 对表达式操作的检查 (`OperandCheck`函数) 该函数用于检查表达式中的操作是否合法, 是类型检查的核心部分。它接受一个 `cact_expr_ptr` (表达式节点指针), 通过操作符类型 `op` 和子表达式 `subexprs` 来判断表达式是否满足语言的语义规则。若表达式为一元操作 (如 `-x`, `!x`), 它要求该操作数不是数组类型 (即不能是 `OP_ARRAY`), 并对不同一元操作符限制支持的基本类型。例如, 取正/负号 (`OP_POS / OP_NEG`) 仅允许 `int`, `float` 或 `char` 类型, 而逻辑非 `OP_NOT` 只允许 `bool` 或 `int` 类型。若不满足这些条件, 将输出错误信息并终止编译。若表达式为二元操作 (如 `a + b`, `a == b`), 同样首先检查两个操作数是否为数组, 若是则报错。接着要求两个操作数的类型一致 (除非是 `&&`, `||`, `==`, `!=` 等操作, 这些允许混合类型如 `bool` 与 `int`), 最后根据不同操作符检查它们所支持的类型集合。例如, `MOD` 只支持 `int` 类型, 比较和算术操作不允许 `bool` 类型等。如果不符合类型要求, 同样报错终止。这一函数确保了语法树中表达式节点的类型组合合法, 是语义分析最核心的功能之一。
 - `enterExp/exitExp`用于检查表达式。
 - `exitExp`在退出表达式规则时, 根据具体情况处理, 这里需要单独考虑布尔常量:
 - 如果表达式是布尔常量 (`BoolConst`), 则创建一个对应的结构体, 通过 `reset`方法让共享指针 `ctx->self`指向它。
 - 否则, 将表达式的返回值设置为 `addExp`的返回值。
 - `enterConstExp/exitConstExp`检查常量类型, 分别考虑常量是数字和布尔常量两种情形, 如果常量的基本类型与声明不一致, 则报错。对于是数字的情形, 前面在 `ConstInitVal`中继承下来的基本类型 `basety`在这里发挥作用, 用于检查。
 - `enterCond/exitCond`用于处理条件表达式。返回值必须是布尔类型且不是数组, 否则报错。
 - `enterLVal/exitLVal`对左值表达式规则进行语义分析。`exitLVal`首先通过符号表中定义的向根追溯查找的方式, 从变量表中检查和提取变量, 并判断变量是否存在, 如果不存在则报错。如果存在, 则继续进行维度检查和类型继承。如果是数组, 先检查维度列表的长度是否超过维度个数, 随后检查每个维度的类型是否是终结符 `int`常量。
 - `enterPrimaryExp/exitPrimaryExp`在退出主表达式时, 根据具体的子表达式类型, 自下而上传递指向子表达式结构体的指针。

- `enterNumber/exitNumber`用于处理数字表达式。在退出数字表达式时，根据数字类型是整数、单精度浮点、双精度浮点三种情况，分别创建对应的表达式结构体，并让`ctx->self`指向它。需要注意的是，对于整数的情形需要指定`int_result`。
- `enterUnaryExp`和`exitUnaryExp`用于处理一元表达式。
- 在退出`UnaryExp`时，根据具体的子表达式处理：
 - 如果是`primaryExp`，则自下而上传递指向子表达式结构体的指针。
 - 如果是一元表达式，构造`self`指向的结构体，并调用`OperandCheck`来针对操作符的操作对象检查。
 - 如果是函数调用，先检查函数表项是否存在，然后根据返回值的类型构造`self`指向的结构体。由于它后续的用法和变量一样，因此这里我们直接将其看做变量处理，`op`设置为`OP_ITEM`。
- 作用域
 - `enterCompUnit`函数:当进入到`CompUnit`时，创建一个新的`scope_node_t`对象，赋值给`root_scope`，用来表示全局作用域。根作用域节点的父节点设为空，并把当前作用域设为`root_scope`。此后，每进入一个`block`就新建一个`scope_t`节点，将当前作用域设为这个新建的作用域节点，并把父指针指向`cur_scope`的旧值。同时调用`addBuiltinFunc`将内联函数，如`print_int`，`print_double`和`get_int`添加到函数表中，防止在测试用例直接调用时报错。
 - `enterBlock/exitBlock`
 - `enterBlock`在进入块规则时，创建一个新的作用域，并将其设置为当前作用域。如果块规则关联着函数参数列表，也就是函数声明作用域的情形，则将函数参数逐一添加到作用域的变量表中。
 - `exitBlock`在退出块规则时，根据实验指导中的简化约定，如果块中有`return`语句或包含`return`的`stmt`，则一定在最后一句。由于这个返回类型的获取与位置有关，可以考虑使用综合属性，按自底向上的方式传递。这种设计也方便了对`if-else`的分支结构返回类型的检查。在该函数中，我们根据指针判断最后一个子节点是否为`stmt`来确定是否存在`return`语句。如果存在，则将块的返回类型设置为最后一个语句的返回类型；否则，将块的返回类型设置为`BTY_VOID`（空类型）。然后退出当前作用域，将作用域切换回父作用域。

```
void SemanticAnalysis::exitBlock(CACTParser::BlockContext *ctx){
    //根据简化，若有return，一定在最后一个语句
    //根据指针判断最后一个子节点是stmt
    if(ctx->stmt().size()!=0 && (void*)(ctx->children[ctx->children.size()-2]) == (void*)(*ctx->stmt().rbegin())){
        ctx->ret_type = (*ctx->stmt().rbegin())->ret_type;
    }
    else{
        ctx->ret_type = BTY_VOID;
    }
    //退出作用域
    cur_scope = cur_scope->parent;
}
```

应用该条件时需要增加检查`ctx->stmt().size()!=0`，否则当`block`中没有`stmt`时，会触发内存泄漏。

- 语句

- `enterStmt_assign/exitStmt_assign` : `enterStmt_assign`和`exitStmt_assign`对赋值语句规则进行语义分析。在进入赋值语句时无操作，而在退出赋值语句规则时，首先进行左值检查，确保左值不是常量。左值是否存在于变量表的检查会在`exitLVal`中完成，这里无需检查。然后调用`OperandCheck`函数进行类型检查。最后，将赋值语句的返回类型设置为`BTY_VOID`。
- `enterStmt_exp/exitStmt_exp` : 在进入和退出表达式语句时被调用。`exitStmt_exp`在退出表达式语句规则时，将表达式语句的返回类型设置为`BTY_VOID`。
- `enterStmt_block/exitStmt_block`用于处理block套block的情形。`enterStmt_block`: 在进入块语句规则时，将该块的函数参数列表指针设置为`nullptr`。`exitStmt_block`: 在退出块语句规则时，自下而上传递子块的返回类型。
- `enterStmt_if/exitStmt_if` `enterStmt_if`无操作。`exitStmt_if` : 在退出条件语句规则时，根据简化的约定，如果条件语句的if-else分支中的语句包含`return`语句，则该`return`语句必定是块的最后一句。并且，我们将没有else分支的情形视为有else分支，但不执行任何操作，其返回类型为`void`。
 - `enterStmt_while/exitStmt_while` `enterStmt_while`无操作。`exitStmt_while` : 这里认为跳出循环后的分支是`void`。和前面提到的简化约定一样，如果while语句的语句部分包含`return`语句，则该语句必须在块的最后。
 - `enterStmt_bcr/exitStmt_bcr`对分支控制语句进行语义分析。`exitStmt_bcr`在退出分支控制语句时，根据具体情况处理返回类型：
 - 如果语句是`return exp`形式，则返回类型由`exp`的类型决定，需要检查`exp`是否是数组，因为`return`不允许返回数组类型，只允许返回基本类型。如果不是数组，则可以把`exp`的类型赋给`ctx->ret_type`。
 - 如果语句不是`return exp`形式，即对应`break`、`continue`或`return`的情形，则返回类型为`BTY_VOID`。

- 函数

- `exitCompUnit`函数：在退出`CompUnit`时需要检查main函数相关的属性。首先在函数表中查询是否存在main函数，如果查询失败，则出现语义错误。如果查询成功，则取函数表中记录函数信息的表项（即value值），继续检查main函数的形参列表和返回值类型。正常情况下main函数的形参列表为空并且返回值为整型，如果不满足这两个条件则报错。
- `enterFuncDef/exitFuncDef`用于在进入和退出函数定义时对其进行语义分析。`locals`定义了属性`fparam_list`，表示形参列表。这里`enterFuncDef`函数会把形参列表的指针传给形参节点和`block`。传给形参节点是为了让它后面负责填写该列表，而传给`block`则是为了让其可以提前获取形参列表以加入作用域，避免重复遍历。

```
void SemanticAnalysis::enterFuncDef(CACTParser::FuncDefContext *ctx){
    //由子节点填写形参列表，指定形参顺序
    int order = 0;
    for(auto fparam: ctx->funcFParam()){
        fparam->fparam_list_ptr = &(ctx->fparam_list);
        fparam->order = order;
        order ++;
    }
}
```

```

    }

    //对于block，由fparam提供形参列表
    ctx->block()->fparam_list_ptr = &(ctx->fparam_list);
}

```

`exitFuncDef`函数在退出函数定义规则时，获取函数的名称和返回类型，检查函数是否已经在符号表中定义，如果未定义则创建函数符号表项并添加到符号表中，最后再检查函数声明的返回类型与函数体的返回类型是否一致。

- `enterFuncType/exitFuncType` `enterFuncType`无操作 `exitFuncType`在退出函数类型时，将函数类型填入函数类型规则的基本类型字段**basety**。这里直接获得的是函数类型的字符串形式，需要借助我们在**cact_types.h**中定义的**map**转化为对应的枚举类型。
- `enterFuncFParam/exitFuncFParam`进入和退出函数参数规则时被调用。
- `exitFuncParam`首先获取函数参数的名称、基本类型和维度信息，根据左括号的数量确定数组的总层数，并将维度长度依次添加到数组维度列表中。对于隐式声明的情形，向数组维度列表中填入0，这样后续可以通过是否为0判断隐式声明。这里使用0是因为样例中不会出现0，正好可以用来作为标记。随后创建函数参数的类型对象，并填入前面获取的内容，然后检查函数参数是否已经在参数列表中定义，如果未定义则创建函数参数项并添加到参数列表中。这里的参数列表正是前面由**enterFuncDef**函数传过来的列表。

• 变/常量声明

由于变量和常量的检查高度相似，因此我们使用模板函数**enter/exitConst_Var_Decl**实现相关功能，在需要使用时调用即可。这里仅以常量为例进行说明：

当在**CACTParser**解析过程中遇到声明语句时，**ANTLR**会调用**enterDecl/exitDecl**函数这两个函数，用于进入和退出声明（包括变量声明和函数声明），但在我们的实现中没有定义具体的操作。由于我们在**enterConst_Var_Decl**函数中的特殊设计，这里无需在**exitDecl**时重复遍历**Def**再来进行类型检查，而是在前面第一次自动遍历**Def**时就已完成。

enterConstDecl/exitConstDecl函数这两个函数在进入和退出常量声明时调用。在进入常量声明时，通过调用**enterConst_Var_Decl**函数来对常量定义进行语义分析，将后续**Def**的基本类型传递向**btype**。该函数在定义时使用了模板**template <typename T1,typename T2>**，在本次调用中，**T1**被替换成**CACTParser::ConstDeclContext**，而**T2**被替换成指向**CACTParser::ConstDefContext**的指针。**constDef()**方法会获取**ctx**中所有匹配**constDef**规则的语法结构，并返回指向它们的指针。

enterBType/exitBType函数用于进入和退出基本类型（**BType**）。在退出时，将基本类型值赋值给**ctx->basety**，并且填写**ctx->passTo**指向的所有节点的**BType**，从而实现基本类型的继承传递，这样后面的函数才能检查类型是否匹配。

• 多维数组

- **enterArrayDims/exitArrayDims**函数用于在进入和退出数组维度**ArrayDims**时进行语义分析。其中**ArrayDims**是为了方便在进入**constDef**和**varDef**时为**constinitval**获取维度数组而设计的。

exitArrayDims函数负责解析数组的维度。它会遍历**ctx->IntConst()**中匹配的所有整数常量，对于每个整数常量，将其文本表示转换为无符号整数并存储在**len**中，这样就得到了维度的长度。随后它

将这些维度长度填入dims_ptr所指向的vector。

- enter/exitConstInitVal函数用于检查常量初始值的合法性，包括基本类型的继承、维度的传递和维度索引的检查。具体而言，enterConstInitVal中会自顶向下继承数组的基本类型basety，在最内层节点(constExp)完成检查。随后，它将basety和维度指针dims_ptr传递给子数组或子元素的初始值。而在exitConstInitVal函数中，实现了对枚举和嵌套两种形式初始值的支持，并且不允许二者混合。该函数会自底向上检查维度，直接推出constExp的dim_index为0，推出{}为1,这两种情况都不需要检查维度。对于其他情况，先检查所有子数组的index是否一致，然后分情况考虑。对最外层数组且无嵌套的情形，应当以列举方式初始化初始化，并且列举的值的个数应该等于数组的元素个数。而对于嵌套形式定义，则从最内层维度开始检查：对于初始值维度大于声明的情况，在该节点报错；对于初始值维度小于声明的情况，则由exitDef负责检查。

4. 中间代码生成(IR_GEN模块)

CACT.g4在中间代码生成中做出的改动

```
stmt
  locals[
    cact_basety_t ret_type,
    //IR
    std::string in_label, // 入点
    std::string out_label, // 出点
    // 传递最终确定层级
    std::string break_label,
    std::string continue_label,
  ]
  : lVal '=' exp ';'                                #stmt_assign
  | (exp)? ';'                                       #stmt_exp
  | block                                           #stmt_block
  | IF '('cond')' lab stmt (go lab ELSE stmt)? lab  #stmt_if
  | lab WHILE '('cond')' lab stmt go lab           #stmt_while
  | (BREAK | CONTINUE | RETURN exp?) ';'          #stmt_bcr
  ;
```

为了保证语法分析的正确性，lab和go的语法规则都是生成空串。此外，对两者定义in_label和out_label属性，在翻译成中间代码时会分别产生跳转语句和代码标签。以if控制流语句为例，对于if语句，条件判断为真则跳转到if后的stmt，因此需要在该stmt前添加标签lab；对于if-else语句，条件判断为真跳转到if后的stmt，为假时跳转到else后的stmt，因此需要在这两个stmt前都要加上lab标签。与此同时，在第一个stmt结束之后要跳转出if语句，所以还要加上go。while控制语句添加标签和跳转语句的原理也类似。.g4文件中lab和go的语法规则和属性值定义如下：

```
lab
  locals[
    std::string in_label,
  ]
  :
  ;
```

```

go
    locals[
        std::string out_label,
    ]
    :
    ;

```

定义IR_GEN宏

在SemanticAnalysis.h头文件中，对中间代码模块进行定义：

```

#ifdef IR_GEN
#define TEMP_PREFIX      '%'
#define ADDR_INFIX      '>'    //表示子数组相对变量的偏移
#define ITEM_INFIX      '<'    //表示子元素相对变量的偏移
#define ARRAY_PLACEHOLDER '$'
#define IMM_PREFIX      '#'
#define LABEL_PREFIX    'L'
#define GLOBAL_PREFIX   '@'
#define LOCAL_PREFIX    '^'
#endif

```

- `IRC_array`: 用于存储完整的中间代码序列。每一条 `IR_code_t` 结构体表示一条 IR 指令，记录了操作码 `IRop`，基本类型 `basety`，以及最多三个字符串形式的操作数或结果（`result, arg1, arg2`）。
- `IR_temp_t` 向量数组：有三个分别用于记录变量信息的数组：`Gvar_array`：全局变量记录（变量名以 @ 开头）；`Lvar_array`：局部变量记录（变量名以 ^ 开头）；`Temp_array`：临时变量记录（变量名以 % 开头）；这些数组以向量形式管理变量，并记录其基本类型、是否为常量、数组长度等元信息。
- `lable_cnt` 和 `new_label` 用于生成唯一的跳转标签，用于控制流 IR（如 `IR_J`, `IR_BNE`, 等）的跳转目标。`len_from_type` 函数用来计算变量的长度，单个变量（`type.arrdims.size()==0`）如 `a`, `b` 的长度为 1。若变量为形如 `c[2][2]` 的多维数组，则需要计算整个数组的长度，即元素个数。该函数计算出的变量长度也需要记录在 `Temp_array/Lvar_array/Gvar_array` 中，以供后续在生成汇编代码时计算函数栈帧大小。

```

#ifdef IR_GEN
std::vector<IR_code_t> IRC_array; // IR存储
size_t label_cnt=0; // 标签序号

size_t len_from_type(cact_type_t type){
    if(type.arrdims.size()==0){
        return 1;
    }
    else{
        size_t product = 1;
        for(int dim: type.arrdims){
            product *= dim;
        }
        return product;
    }
}

```



```

    }
}

std::string newLabel(){
    std::string label_name = LABEL_PREFIX + std::to_string(label_cnt);
    label_cnt++;
    return label_name;
}

```

- addIRC()函数：用于构造中间代码

```

void addIRC(IR_op_t IRop, cact_basety_t basety=BTY_UNKNOWN, std::string
result="", std::string arg1="", std::string arg2=""){
    IRC_array.push_back((IR_code_t){IRop, basety, result, arg1, arg2});
}

```

它支持默认参数，IR_ASSIGN：赋值指令，一般使用 result, arg1; IR_ADD 等算术操作：使用 result, arg1, arg2; IR_LABEL, IR_FUNC_BEGIN：只使用 result 表示标签名或函数名 该方法是整个 IR 生成过程中最核心的函数，确保所有中间代码统一进入 IRC_array。

- 辅助输出函数(并不是解释器)printIRCode()和printAllIRCodes() printIRCode() 以格式化方式打印一条 IR 指令的操作码、类型、操作数等； printAllIRCodes() 遍历 IRC_array 并逐条调用 printIRCode()，完整打印当前生成的 IR 序列。这是中间代码生成与调试的标准实践，有助于直观验证语义分析和翻译结果，并不是最终的解释器打印结果。

```

// 测试用打印IRcode
// 打印单个IR指令的函数
void printIRCode(const IR_code_t& code) {
    std::cout << "OP: " << typeutils.IRop_to_str[code.IRop]
    << " | Type: " << typeutils.basety_to_str[code.basety]
    << " | Result: " << (code.result.empty() ? "NONE" :
code.result)
    << " | Arg1: " << (code.arg1.empty() ? "NONE" : code.arg1)
    << " | Arg2: " << (code.arg2.empty() ? "NONE" : code.arg2)
    << std::endl;
}
// 打印整个IR数组的函数
void printAllIRCodes() {
    std::cout << "\n=== IR Code Dump (" << IRC_array.size() << "
instructions) ===" << std::endl;
    for (size_t i = 0; i < IRC_array.size(); ++i) {
        std::cout << "[" << i << "] ";
        printIRCode(IRC_array[i]);
    }
    std::cout << "=== End of IR Dump ===\n" << std::endl;
}

```

语义分析过程中如何生成中间代码(SemanticAnalysis.cpp)

- 变量/常量的定义
 - exitConst_Var_Def函数

在离开ConstDef或VarDef节点时，首先判断所定义的变量是否为全局变量，如果是，则生成对应的中间代码中IRop为G_ALLOC，否则为L_ALLOC。利用变量类型和len_from_type计算变量长度后，使用newGvar或newLvar将该变量添加到array中存储，并得到变量名。将变量名写入符号表中，以便在后续的中间代码生成中直接使用。相关代码如下：

```
bool is_global = cur_scope == symbol_table.root_scope;
IR_op_t IRop;
size_t len;
std::string IR_name;
if(is_global){
    IRop = IR_G_ALLOC;
    len = len_from_type(ctx->type);
    IR_name = newGvar(ctx->basety, is_const, len);
}
else{
    IRop = IR_L_ALLOC;
    len = len_from_type(ctx->type);
    IR_name = newLvar(ctx->basety, is_const, len);
}

symbol_table.var_table[(name_scope_t){name,cur_scope}] =
(var_symbol_item_t){.type=ctx->type,.IR_name=IR_name};
```

如果在检查constDef或varDef节点时发现了constInitVal子节点，说明在声明该变量时还同时对它进行了初始化，因此在获得初始值后再加上立即数前缀IMM_PREFIX便可生成一条声明该变量的中间代码：

```
std::string initval = IMM_PREFIX + ctx->constInitVal()->value_list;
addIRC( IRop, ctx->basety,IR_name,initval);
```

- exitConstInitVal函数

该函数在上一个实验中实现了对多维数组枚举和嵌套初始化定义检查的支持。在生成多维数组初始化的中间代码时，如果在定义中出现元素的缺失，需要用占位符ARRAY_PLACEHOLDER(\$)来填充。例如比如嵌套定义`a[3][2]={ {}, {1} }`，那么对于最里面花括号，就要填充成2个元素，对于外层的花括号，要由三个拼接，并且每个都要求有2个长，所以最后在中间代码中被初始化成`$$,1,$,$`。我们用hold_len表示每一个嵌套层需被填充的长度，len表示实际定义的元素数量，如果hold_len>len，则需要增加占位符\$。

- 函数的定义

- enterFuncDef和exitFuncDef函数

进入FuncDef节点后，此时正在进行一个函数的定义，因此需要生成一条IR_FUNC_BEGIN的中间代码。离开FuncDef节点说明函数的定义已经结束，需要生成一条IR_FUNC_END的中间代码

- enterBlock函数

如果block是函数在定义时的block，则需要在符号表中加入函数定义时用到的形参名字IR_name，方便后续生成中间代码中对变量名的直接使用。

```
std::string IR_name = newLvar(basety); //使用缺省参数 isconst=false, len=1
(*ctx->fparam_list_ptr)[i].IR_name = IR_name;
var_symbol_item_t item = (var_symbol_item_t)
{.type=fparam.type, .IR_name=IR_name};
```

- exitStmt_bcr函数

在函数的定义中，最后一句可能是return返回语句，因此如果stmt_bcr的语法规则中生成了RETURN节点，会生成一条IRop为IR_RETURN的中间代码语句，并将ret_type和RETURN token后exp节点的result_name属性添加到该代码语句中。

- 函数的调用

- exitUnaryExp函数

unaryExp : Ident LeftParen (funcRParams)? RightParen 定义了函数调用的语法规则，所以在离开exitUnaryExp节点和检查传参过程中各变量的类型和数量是否与被调用函数的形参列表一致后，若检查无问题，则会为每一个参数生成IRop为IR_PARAM的中间代码，并将该参数的基本类型basety和变量名exp_name填入到代码中。

完成IR_PARAM传参语句后，生成IR_CALL中间代码。此外还需要注意的是，如果函数的返回值不是void类型，我们还需要再分配一个临时变量来存储该函数调用的返回值：

```
if(basety==BTY_VOID){
    addIRC( IR_CALL,
            BTY_VOID,
            func_name);
}
else{
    ctx->result_name = newTemp(basety);
    addIRC( IR_L_ALLOC,
            basety,
            ctx->result_name);
    addIRC( IR_CALL,
            basety,
            func_name,
            ctx->result_name);
}
```

- 控制流语句的翻译

.cact源码中的控制流语句以if/if-else/while为主，这里以while语句为例阐述生成相应中间代码的过程。

- enterStmt_while函数

根据语法规则 `stmt : lab WHILE LeftParen cond RightParen lab stmt go lab`，我们需要在中间代码中生成3个lab对应的代码标签，分别为start_label，block_start和end_label。如果cond为真，控制需要跳转到block_start的代码位置，如果为假，则会跳出while循环，控制转移到end_label的代码位置。因此 `ctx->cond()->true_label` 和 `ctx->cond()->false_label` 分别赋为block_start和end_label。此外，如果stmt执行完毕，需要返回while中的条件语句进行是否进行下一轮循环的判断，因此stmt后的go会被翻译成跳转到start_label处代码的跳转指令。如果stmt是一个break语句，则结束while循环，stmt节点的break_label属性被赋为end_label；如果stmt是一个continue语句，则控制直接转换到while的条件语句处，stmt节点的continue_label属性被赋值为start_label

```
void SemanticAnalysis::enterStmt_while(CACTParser::Stmt_whileContext
*ctx){
    #ifdef IR_gen
        std::string start_label = newLabel();
        std::string end_label = newLabel();
        std::string blk_start = newLabel();
        ctx->cond()->true_label = blk_start;
        ctx->cond()->false_label = end_label;
        ctx->lab()[0]->in_label = start_label;
        ctx->lab()[1]->in_label = blk_start;
        ctx->lab()[2]->in_label = end_label;
        ctx->go()->out_label = start_label;
        //沿子节点传递
        ctx->stmt()->break_label = end_label;
        ctx->stmt()->continue_label = start_label;
    #endif
}
```

- enterLAndExp/enterLOrExp/enterRelExp函数

对于控制流中的条件语句cond，其条件表达式的操作符可能为&&、||或==，分别对应着LAndExp、LOrExp和RelExp的情况，这里以enterLAndExp函数为例说明代码翻译的过程：

`lAndExp : lAndExp1 AND lab eqExp`语法规则定义了&&表达式。对于lAndExp1，如果表达式为真，则控制需要跳转到eqExp前的lab代码标签继续进行条件判断，若eqExp条件判断也为真，则整个lAndExp表达式为真，控制流跳转到lAndExp的true_label处，即 `ctx->true_label`。若eqExp条件判断为假，整个lAndExp表达式为假，控制流跳转到lAndExp的false_label处，即 `ctx->false_label`。如果lAndExp1表达式为假，则整个lAndExp表达式为假，控制跳转到lAndExp的false_label处，即 `ctx->false_label`。代码实现逻辑如下：

```
void SemanticAnalysis::enterLAndExp(CACTParser::LAndExpContext *ctx){
    #ifdef IR_gen
        if(ctx->lAndExp()!=nullptr){ //lAndExp AND eqExp
```

```

//部分为true不需要特别处理，继续向下即可，不必和OR一样加额外标签
std::string and_label = newLabel();
ctx->lab()->in_label = and_label;
ctx->lAndExp()->true_label = and_label;
ctx->lAndExp()->>false_label = ctx->>false_label;
ctx->eqExp()->true_label = ctx->true_label;
ctx->eqExp()->>false_label = ctx->>false_label;
ctx->eqExp()->has_label = true;
}
else{//eqExp
    ctx->eqExp()->true_label = ctx->true_label;
    ctx->eqExp()->>false_label = ctx->>false_label;
    ctx->eqExp()->has_label = true;
}
#endif
}

```

- 数组左值和子数组地址的获得

- exitLVal函数

1) 如果表达式的操作类型为OP_ITEM，则要求获得数组的左值，所以需要根据数组的维度、给定的index和元素类型来计算目标位置相对数组基地址的偏移量。具体代码逻辑如下所示：

```

#ifdef IR_GEN
//单位元素的字节偏移，语义分析不出现PTR，只有IR传参的时候可能使用
int base_size;
switch(iter_type.basety){
    case BTY_BOOL:
        base_size = 1; break;
    case BTY_INT: case BTY_FLOAT:
        base_size = 4; break;
    case BTY_CHAR:
        base_size = 8; break;
    default:
        break;
}

//为了充分利用公共子表达式对维度的计算，每个计算结果都用新的变量名存储，防止result和arg相同
std::string index_offset; //当前维度的字节偏移
std::string old_offset; //之前累加
std::string byte_offset; //本轮后累加
for(int i=0; i<layer; i++){
    int subproduct = 1;
    for(int j=i+1; j<dim_size; j++){
        subproduct *= iter_type.arrdims[j];
    }
    std::string times = IMM_PREFIX +
std::to_string(subproduct*base_size);
    index_offset = newTemp(BTY_INT);
    addIRC(IR_L_ALLOC, BTY_INT, index_offset);
}

```

```

        addIRC(IR_MUL,BTY_INT,index_offset,exp_list[i]-
>result_name,times);
        if(i==0){
            byte_offset = index_offset;
        }
        else{
            old_offset = byte_offset;
            byte_offset = newTemp(BTY_INT);
            addIRC(IR_L_ALLOC,BTY_INT,byte_offset);
            addIRC(IR_ADD,BTY_INT,byte_offset,old_offset,index_offset);
        }
    }

#endif

```

2) 如果表达式的操作类型为OP_ARRAY，则要求获得子数组的地址，子数组的地址相对于数组基地址的偏移的计算与上述情况相同，但在生成中间代码时要用不同的前缀区分这两种情况：

```

#ifdef IR_GEN
//形如abc>%2 或abc<%2
char infix = (op==OP_ARRAY) ? ADDR_INFIX : ITEM_INFIX;
ctx->result_name = IR_name + infix + byte_offset; // g4
#endif

```

IR解释器的设计

解释器的核心任务是：遍历执行由编译器前端生成的三地址码 IR 序列。在执行过程中，它维护一个程序计数器 pc、函数调用栈、变量值映射，并根据不同的 IR 操作码执行对应语义（赋值、运算、跳转、函数调用等），模拟一个完整的运行环境。

大致思路为pc作为中间指令的标签名，第一次扫描确定标签名和文本上的中间代码顺序，第二次得到实际执行的中间代码顺序。

- 第一次预扫描：记录每个标签或函数名对应的 pc 地址，便于跳转或函数调用时快速定位执行点。该表存储在 labels 中，键为标签名，值为一个 lab{begin, end} 结构体。

```

while (pc < IRC_array.size()) {
    switch (ir.IRop) {
        case IR_FUNC_BEGIN:
        case IR_FUNC_END:
        case IR_LABEL:
            labels[ir.result].begin / end = pc;
    }
}

```

- 第二次扫描：是解释器执行过程的主循环，再次从 `pc = 0` 开始，在 `while (pc < IRC_array.size() && running)` 中进行逐条 IR 指令解释执行。每条指令根据其类型（操作码 `IRop`）进入 `switch` 分支处理，操作包括：
 - 函数调用/返回：`IR_FUNC_BEGIN`, `IR_FUNC_END`, `IR_CALL`, `IR_RETURN` 赋值与算术运算：`IR_ASSIGN`, `IR_ADD`, `IR_SUB` 等 逻辑运算：`IR_AND`, `IR_OR`, `IR_NOT`, `IR_NEG` 控制流：`IR_J`, `IR_BEQ`, `IR_BNE`, `IR_BLT` 等 条件跳转 变量分配与初始化：`IR_G_ALLOC`, `IR_L_ALLOC` 执行时，解释器解析变量名中的偏移信息（如数组元素访问），调用 `getValue()` 获取操作数值，再调用 `setValue()` 存储结果。

示例代码：

```
pc = 0;
int val;
int length;
int index;
bool running = true;
while (pc < IRC_array.size() && running) {
    const IR_code_t& ir = IRC_array[pc];
    std::string str1, str2;
    std::string name = ir.result;

    std::cout << "[" << pc << "]" ";
    printIRCode(ir);

    int pos = name.find('<');
    if(pos == std::string::npos){
        str1 = name;
        str2 = "";
        index = 0;
    }
    else{
        str1 = name.substr(0, pos);
        str2 = name.substr(pos+1);
        index = getValue(str2)/4;
    }

    switch (ir.IRop) {
        case IR_FUNC_BEGIN:
            if(ir.result == "main"){
                funcstack.push_back((func){ .param="", .ra=0,
                .funcname=ir.result, .returnval=""});
                pc++;
                break;
            }
            else if(funcstack.empty()){
                pc = labels[ir.result].end + 1;
                break;
            }
            else{
                pc++;
                break;
            }
    }
}
```


举例：当出现程序控制流指令(如:IR_J)时：

```
case IR_J:
    pc = labels[ir.result].begin + 1;
    break;
```

此时labels[ir.result].begin即是最终结果指向的标签名，即指令IR_label，此时再+1就是下一条指令的pc值。

- `getValue(std::string name)`用于从变量名 `name` 中解析出实际的数值。变量名可以是一个普通变量、数组元素、临时变量、立即数，甚至可能嵌套表达式形式如 `%2<%3`。函数首先通过 `find('<')` 判断是否包含数组偏移部分。若存在，`str1` 取主变量名（如 `%2`），`str2` 为偏移表达式（如 `%3`），然后通过递归调用 `getValue(str2)` 求偏移量，并按 4 字节对齐（/4）确定索引位置。根据变量名前缀判断其类型：`@` 表示全局变量，从 `gvar_vals` 获取值；`%` 表示临时变量，从 `temp_vals` 获取；`^` 表示局部变量，从 `lvar_vals` 获取；`#` 表示立即数，去掉前缀并转换为整数；若为局部变量且 `lvar_vals[name]` 为空（即没有明确赋值），说明可能是函数参数，在这种情况下会退回到当前函数栈帧中查找其传入参数 `param`，并通过 `getValue(funcstack.back().param)` 递归获取实参值。该函数通过统一接口实现对不同作用域变量、数组元素、立即值等的透明访问，是 IR 执行中读取操作数的核心方法。

```
int getValue(std::string name){
    int pos = name.find('<');
    std::string str1,str2;
    if(pos == std::string::npos){
        str1 = name;
        str2 = "";
    }
    else{
        str1 = name.substr(0,pos);
        str2 = name.substr(pos+1);
    }
    if (name[0] == '@' && str2 == "")
        return gvar_vals[name][0];
    else if(name[0] == '@')
        return gvar_vals[str1][getValue(str2)/4];

    if (name[0] == '%' && str2 == "")
        return temp_vals[name][0];
    else if(name[0] == '%')
        return temp_vals[str1][getValue(str2)/4];

    if (name[0] == '^' && str2 == ""){
        if(lvar_vals[name].empty())
            return getValue(funcstack.back().param);
        return lvar_vals[name][0];
    }
    else if(name[0] == '^')
        return lvar_vals[str1][getValue(str2)/4];

    if (name[0] == '#') return std::stoi(name.substr(1),nullptr,0);
```

```

        return 0;
    }

```

- `setValue(std::string name, int val, int index)`: `setValue` 用于将计算结果存储到变量指定位置。 `setValue` 接收变量名 `name`、要写入的值 `val` 和目标数组位置索引 `index`。变量名通过前缀识别其所属值表（@→全局，^→局部，%→临时），然后根据当前变量的值表中是否已有该位置：

如果该索引处尚未存在值（即超出当前 `vector` 长度），则使用 `push_back` 添加新值；

否则，直接对已有位置进行覆盖更新。

该函数封装了变量值的写入逻辑，支持普通变量和数组变量的统一写入，确保解释器执行过程中变量状态始终保持一致。数组写入时依赖外部对 `index` 的计算，因此常与 `getValue()` 联动使用。

```

void setValue(std::string name, int val, int index) {
    if (name[0] == '@' && gvar_vals[name].size() <= index) {
        gvar_vals[name].push_back(val);
    }
    else if (name[0] == '@') {
        gvar_vals[name][index] = val;
    }

    if (name[0] == '%' && temp_vals[name].size() <= index) {
        temp_vals[name].push_back(val);
    }
    else if (name[0] == '%') {
        temp_vals[name][index] = val;
    }

    if (name[0] == '^' && lvar_vals[name].size() <= index) {
        lvar_vals[name].push_back(val);
    }
    else if (name[0] == '^') {
        lvar_vals[name][index] = val;
    }
}

int getLength(std::string name) {
    if (name[0] == '@') return
Gvar_array[std::stoi(name.substr(1)), nullptr, 0].length;
    if (name[0] == '%') return
Temp_array[std::stoi(name.substr(1)), nullptr, 0].length;
    if (name[0] == '^') return
Lvar_array[std::stoi(name.substr(1)), nullptr, 0].length;
}

```

- `printIRCode(const IR_code_t& code)`: 最终的中间代码打印函数 输出结果类似如下：

```
[12] OP: MUL | Type: int | Result: %5 | Arg1: %3 | Arg2: #2
```

其中12就是pc值。打印按照IR设计的格式进行排序，用|做分割。

```
void printIRCode(const IR_code_t& code) {
    std::cout << "OP: " << typeutils.IRop_to_str[code.IRop]
        << " | Type: " << typeutils.basety_to_str[code.basety]
        << " | Result: " << (code.result.empty() ? "NONE" : code.result)
        << " | Arg1: " << (code.arg1.empty() ? "NONE" : code.arg1)
        << " | Arg2: " << (code.arg2.empty() ? "NONE" : code.arg2)
        << std::endl;
}
};
```

解释器的编写较为粗糙，很多情况下因为测试样例的特殊性做出了简化。比如，由于functional下的样例中只有32包含浮点数，且只有32中定义的函数具有两个参数，目前编写的解释器只支持储存int类型的值，从代码中可以看出gvar_vals等键值对的值都只是一个std::vector<int>，显然存不了浮点数，解释器也就不支持浮点数的计算。解释器目前也不支持传递两个参数。因此解释器目前不能运行32样例。

5. 主程序(main.cpp)

main.cpp中的逻辑与上一次实验大致相同。由于本次实验新增的功能，我们需要先实例化符号表SymbolTable和用于语义分析的SemanticAnalysis。随后，程序会打开一个文件流，并将其传递给ANTLRInputStream以获取输入。然后，它创建CACTLexer和CACTParser对象，用来解析输入，并创建语法分析树。由于我们实现的是listener模式，因此需要使用walk方法遍历树，并调用SemanticAnalysis中的函数来执行语义分析并生成中间代码，然后再调用解释器解释执行中间代码。

```
Analysis listener;
tree::ParseTreeWalker::DEFAULT.walk(&listener, tree);

//semantic
tree::ParseTreeWalker::DEFAULT.walk(&semantic_analysis, tree);

//std::cout << "True" << std::endl;
semantic_analysis.printAllIRCodes();

Interpreter interpreter(semantic_analysis.IRC_array,
                        semantic_analysis.Temp_array,
                        semantic_analysis.Lvar_array,
                        semantic_analysis.Gvar_array);

interpreter.run();
return 0;
```

测试运行

我们测试了functional下的33个样例，除了32.cact解释器不支持外，其他均可以输出结果。测试并改善解释器以及中间代码后，最终只有29.cact的输出出现了部分偏差，其他样例均可以输出正确结果。29样例不是完全正确的原因是目前中间代码不支持把关系运算如<， ==等的结果作为值来使用，只能根据该算式的ture或false来决定是否跳转，也就是只会生成BEQ、BLT等中间代码。如(3 > 4 == 0)就判断不了了。

main.cpp中第61行semantic_analysis.printAllIRCodes()用于打印该样例生成的所有中间代码，IRgen.h的第50、51行是用于按顺序打印实际执行的中间代码。可以根据需要决定是否要打印这些中间代码。如果只想进行测试的话，可以把上述代码注释后直接在build里./test.sh，会直接运行functional中的全部样例。（不注释掉的话中间代码打印的太多了看不清）

总结

实验结果总结

在本实验中，我们完成了语义分析、中间代码生成并编写了一个简单的解释器运行中间代码。

分成员总结

俞博涵：本次实验中我参与了部分语义分析的编写工作，参与了IR的设计，构建了解释器的大体框架，以及进行了部分bug的修改。通过本次实验，更深入理解了编译器的语义分析和中间代码生成过程，重温了代码运行的基本流程，学习并实际练习C++使我更加熟悉这一强大的语言。

杜璵：本次实验中我参与了语义分析的编写工作，参与了IR设计的讨论，具体实现了解释器的主要功能，修改了部分bug。本次实验让我对c++更加了解，对编译器遍历语法树进行语义分析并生成中间代码有了更深刻的认识。